ChatGPT

BrewLite GUIDE-V2 C – Cart: Hướng dẫn cầm tay chỉ việc

Tóm tắt điều hành

Module **C – Cart** phụ trách trọn vẹn **Task 5 – Giỏ hàng**: nhận cấu hình sản phẩm từ màn hình Catalog, lưu giỏ trên trình duyệt bằng Zustand, cho tăng/giảm số lượng, xóa món, tính tổng tiền hiển thị, giữ giỏ sau refresh, hiển thị badge trên header và chuyển người dùng sang `/checkout`. Đây đúng với phạm vi trong guide C gốc và Product Backlog của BrewLite. filecite turn0file14 filecite turn0file9

Điểm quan trọng nhất của GUIDE-V2 là phân biệt **"giá hiển thị ở Cart"** và **"giá thật dùng để tạo Order"**:

> **Frontend Cart được phép tính giá để hiển thị, nhưng không bao giờ là nguồn giá đáng tin cậy. Backend Ordering phải tính lại giá từ `productId + size + toppings + qty`.**

Guide E gốc đã giao `POST /orders` cho Ordering, nhận các thuộc tính cấu hình món và tự tính tổng bằng pricing logic; do đó Cart **không được gửi `unitPrice` rồi yêu cầu backend tin giá đó**. filecite turn0file12

BrewLite hiện đã qua GATE-01 ở mức platform: image backend/frontend build được và ba container backend, frontend, PostgreSQL khởi động thành công. Vì vậy người C **không dựng lại project, không sửa Docker và không sửa Prisma schema để tạo bảng Cart**. filecite turn0file8

Kết quả cuối cùng của người C phải tạo ra luồng:

```
Product Detail
      ↓
 Thêm vào giỏ
      ↓
Zustand Cart + localStorage
      ↓
 /cart
      ↓
 /checkout
      ↓
Ordering tạo Order
      ↓
Payments thanh toán
      ↓
thành công → clear cart
thất bại   → giữ cart
```

Đây cũng khớp với luồng nghiệp vụ chính của đề BrewLite: chọn sản phẩm → thêm vào giỏ → xem giỏ và tổng tiền → đăng nhập nếu cần → thanh toán → xác nhận đơn. filecite turn0file9

Tên file PDF cuối cùng đề xuất:

```
BrewLite_GUIDE-V2_C-Cart_2026-10-15.pdf
```

Manifest đi kèm:

```
BrewLite_GUIDE-V2_C-Cart_2026-10-15.manifest.json
```

Phạm vi và ranh giới ownership

Người C thực sự sở hữu gì?

Guide C gốc chỉ định bốn vùng chính: Zustand store, trang `/cart`, pricing frontend và header badge. filecite turn0file14 Với Foundation V1 hiện tại, nên cụ thể hóa thành bảng sau.

File	Quyền của C	Làm gì
`frontend/store/cart.ts`	**OWN / EDIT**	Cart state, actions, persist
`frontend/lib/pricing.ts`	**OWN / EDIT**	Giá preview phía frontend
`frontend/app/cart/page.tsx`	**OWN / EDIT**	Trang Cart
`frontend/components/cart-link.tsx`	**NEW / OWN**	Badge giỏ hàng dạng Client Component
`frontend/app/layout.tsx`	**EDIT NHẸ**	Gắn `CartLink` vào header
`frontend/app/products/[id]/page.tsx`	**B OWN**	Không tự ý sửa; B gọi `add()` của C
`frontend/app/checkout/page.tsx`	**F/G OWN**	C chỉ cung cấp `items`, `total()`, `clear()`
`backend/src/orders/*`	**E OWN**	Không sửa
`backend/src/payments/*`	**F OWN**	Không sửa
`backend/src/common/pricing.ts`	**E/shared contract**	C đọc để đối chiếu, không tự đổi
`backend/prisma/schema.prisma`	**KHÔNG C OWN**	Cart không cần bảng DB

Việc tạo `cart-link.tsx` riêng thay vì biến toàn bộ `app/layout.tsx` thành Client Component là cách an toàn hơn với Next.js App Router: các thành phần cần state, event handler hoặc browser API như `localStorage` phải chạy phía client; Next.js cho phép giữ layout phía server và đặt một Client Component nhỏ bên trong. [1]

B, C, E và F chia nhau như thế nào?

Đây là bảng **bắt buộc cả nhóm hiểu giống nhau**, vì Product Detail, Cart, Checkout và Payment nối sát nhau.

Việc	B – Catalog	C – Cart	E – Ordering	F – Payments
Lấy dữ liệu sản phẩm	**Chủ trì**	Đọc dữ liệu B cung cấp	Đọc DB khi tạo order	Không
Chọn size/ topping	**Chủ trì UI**	Nhận kết quả	Validate/tính lại	Không
Nút “Thêm vào giỏ”	Render nút + gọi C	**Cung cấp** `add()`	Không	Không
Lưu giỏ	Không	**Chủ trì**	Không	Không
Thay qty/xóa món	Không	**Chủ trì**	Không	Không
Giá preview	Có thể gọi helper C	**Chủ trì frontend pricing**	Không	Không
Giá chính thức	Không	**Không có quyền quyết định**	**Chủ trì**	Dùng amount của Order
Kiểm tra stock	Hiển thị thông tin	Không reserve	**Chủ trì**	Không
Tạo Order	Không	Chỉ chuyển data sang checkout	**Chủ trì**	Không
Thanh toán	Không	Không	Quản lý Order	**Chủ trì**
Clear cart	Không	**Cung cấp** `clear()`	Không	Gọi sau payment success
Payment fail	Không	Giữ state	Có thể đổi trạng thái Order	**Báo lỗi, không clear Cart**

Guide B yêu cầu màn hình chi tiết có chọn size/topping và nút “Thêm vào giỏ”; guide C sở hữu Zustand cart; guide E sở hữu tạo Order và tính lại tổng tiền; guide F quy định chỉ xóa giỏ sau khi thanh toán thành công và giữ nguyên giỏ khi thất bại. filecite turn0file15 filecite turn0file14 filecite turn0file12 filecite turn0file11

Quy tắc merge: B không tự viết một Cart store thứ hai; C không tự sửa Product Detail của B; F không tự viết logic cart mới. Tất cả cùng dùng `useCartStore` của C.

Contract dữ liệu, state, pricing và bảo mật

Cart không có backend API riêng

Theo yêu cầu BrewLite, Task 5 mô tả Cart là state frontend bằng Zustand/Context; danh sách endpoint tối thiểu cũng **không có** `/cart` **hoặc** `/carts`. Các API chính gồm `/products`, `/orders`, `/`

`payments`, `/auth`. Vì vậy GUIDE-V2 C không tạo `CartController`, `CartService`, model `Cart` hay bảng `Cart`. filecite turn0file9

Cart trong V1 là:

```
Browser
 └── Zustand
      └── localStorage
```

không phải:

```
Browser → POST /cart → PostgreSQL
```

Nếu sau này muốn cart đồng bộ nhiều thiết bị thì đó là backlog phiên bản sau, không phải MVP hiện tại.

`CartItem` chuẩn của C

Guide gốc yêu cầu mỗi item chứa:

```
productId
name
size
toppings
qty
unitPrice
```

filecite turn0file14

Nên khóa type V2 như sau trong:

```
frontend/store/cart.ts
```

```ts
export type CartSize = 'S' | 'M' | 'L';

export interface CartItem {
  productId: string;
  name: string;
  size: CartSize;
  toppings: string[];
  qty: number;

  /**
   * Giá 1 ly đã gồm:
   * basePrice + phụ thu size + topping.
   *
```

```
   * CHỈ dùng để hiển thị frontend.
   * Backend tuyệt đối không tin giá trị này.
   */
  unitPrice: number;
}
```

Không cần lưu `total`, vì:

```
lineTotal = unitPrice * qty;
```

và:

```
cartTotal = sum(lineTotal);
```

Không nên persist các function Zustand; chỉ persist `items`.

Hai món được coi là cùng một dòng khi nào?

Không thể chỉ so `productId`.

Ví dụ:

```
Cappuccino / M / Kem
```

và:

```
Cappuccino / L / Kem + Trân châu
```

là hai dòng khác nhau.

Tạo helper:

```
function normalizeToppings(toppings: string[]): string[] {
  return [...new Set(toppings)].sort();
}

export function cartLineKey(item: {
  productId: string;
  size: CartSize;
  toppings: string[];
}): string {
  return [
    item.productId,
    item.size,
    normalizeToppings(item.toppings).join(','),
```

```
  ].join('|');
}
```

Khi `add()` nhận món có cùng:

```
productId + size + tập toppings
```

thì tăng `qty`.

Khác một trong ba yếu tố thì tạo dòng mới.

Đây là **quy ước V2 đề xuất**, vì guide gốc chưa nói rõ cách nhận diện hai cấu hình của cùng một Product. Nó cần được ghi vào manifest dưới dạng assumption.

Zustand store phải có gì?

`frontend/store/cart.ts` nên expose contract duy nhất:

```
interface CartState {
  items: CartItem[];

  add: (item: CartItem) => void;

  remove: (
    productId: string,
    size: CartSize,
    toppings: string[],
  ) => void;

  setQty: (
    productId: string,
    size: CartSize,
    toppings: string[],
    qty: number,
  ) => void;

  clear: () => void;

  total: () => number;
  count: () => number;
}
```

Trong đó:

```
total() = tổng tiền
count() = tổng qty, không phải số dòng
```

Ví dụ:

```
Cappuccino x2
Trà đào x3
```

Badge phải là:

```
5
```

không phải `2`.

Guide gốc yêu cầu badge thể hiện số sản phẩm trong giỏ; tổng `qty` là diễn giải phù hợp hơn với hành vi giỏ hàng thông thường. filecite turn0file14

Persistence

Dùng Zustand `persist`:

```ts
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      // ...
    }),
    {
      name: 'brewlite-cart-v1',
      version: 1,
      storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({
        items: state.items,
      }),
    },
  ),
);
```

Zustand chính thức hỗ trợ `persist` để giữ state qua reload và mặc định có thể dùng `localStorage`; middleware cũng hỗ trợ `version`, `partialize`, migration và custom storage. [2]

Không persist:

```
total
count
loading
```

```
error
checkout state
payment state
```

vì các giá trị đó hoặc có thể tính lại, hoặc thuộc module khác.

Validation trong Cart

Khóa rule C như sau:

Field	Rule
`productId`	string không rỗng
`name`	string không rỗng, chỉ để hiển thị
`size`	`S`, `M`, hoặc `L`
`toppings`	array string, loại duplicate
`qty`	integer từ `1` đến `99`
`unitPrice`	integer `>= 0`, frontend display only

`99` là **giới hạn V2 đề xuất**, không phải yêu cầu đề bài. Backend Ordering vẫn phải validate request của chính nó bằng DTO; NestJS khuyến nghị validate payload đầu vào bằng `ValidationPipe`/DTO thay vì tin dữ liệu client. [3]

Ở UI:

```
qty = 1
↓
nút "-" disabled

qty = 99
↓
nút "+" disabled
```

Không dùng `qty = 0` để đại diện dòng tồn tại. Xóa món phải qua `remove()`.

Pricing frontend

Guide C gốc khóa công thức:

```
Giá dòng =
(
  giá gốc
  + phụ thu size
  + phụ thu topping
```

```
)
× số lượng
```

với:

```
S = +0
M = +5.000
L = +10.000

mỗi topping = +5.000
```

filecite turn0file14

`frontend/lib/pricing.ts`:

```ts
import type { CartSize } from '@/store/cart';

export const SIZE_SURCHARGE: Record<CartSize, number> = {
  S: 0,
  M: 5_000,
  L: 10_000,
};

export const TOPPING_SURCHARGE = 5_000;

export function calculateUnitPrice(
  basePrice: number,
  size: CartSize,
  toppings: string[],
): number {
  const uniqueToppings = [...new Set(toppings)];

  return (
    basePrice +
    SIZE_SURCHARGE[size] +
    uniqueToppings.length * TOPPING_SURCHARGE
  );
}

export function calculateLineTotal(
  unitPrice: number,
  qty: number,
): number {
  return unitPrice * qty;
}
```

Ví dụ:

```
Base price        35.000
Size M            +5.000
2 toppings       +10.000
------------------------
unitPrice          50.000

qty = 3
lineTotal         150.000
```

Ai quyết định giá cuối cùng?

E – Ordering.

Guide Ordering nói `POST /orders` nhận:

```
productId
size
toppings
qty
```

và backend tính tổng tiền bằng pricing logic. filecite turn0file12

Do đó C **không gửi**:

```
{
  "unitPrice": 1000,
  "total": 1000
}
```

rồi mong server tin.

Người dùng có thể mở DevTools và sửa `localStorage`; ngay cả Zustand cũng cảnh báo dữ liệu deserialize từ persistence không tự động được runtime-validate, nên persisted state có thể cũ, hỏng hoặc bị sửa. [4]

Contract đúng là:

```
{
  "items": [
    {
      "productId": "PRODUCT_ID",
      "size": "M",
      "toppings": ["PEARL"],
      "qty": 2
    }
```

```
  ]
}
```

Backend E lấy Product thật từ DB và **tính lại giá từ đầu**.

Concurrency và tồn kho

Cart **không giữ chỗ stock**.

Một người có thể thêm 5 Cappuccino vào Cart lúc 10:00, nhưng tới 10:10 stock đã thay đổi. Vì vậy việc item nằm trong `localStorage` **không có nghĩa món vẫn còn hàng**.

Task 10 của BrewLite yêu cầu Ordering kiểm soát đặt đồng thời bằng transaction + optimistic locking; guide E cũng giao stock/version cho module Ordering. filecite turn0file9 filecite turn0file12

Prisma mô tả optimistic concurrency control bằng concurrency token như trường `version`: đọc record, thử update với version mong đợi và phát hiện conflict nếu record đã đổi trước khi ghi. [5]

Vì vậy:

```
C không check stock lần cuối.
C không decrement stock.
C không cập nhật Product.version.
C không mở Prisma transaction.
```

Nếu `POST /orders` trả conflict vì stock không đủ:

```
KHÔNG clear Cart
```

mà checkout phải báo người dùng sửa Cart/thử lại.

Các bước triển khai cầm tay chỉ việc

Chuẩn bị branch

Từ repository sạch:

```
git checkout main
git pull
git checkout -b feature/c-cart
```

Trước khi code:

```
node scripts/verify-foundation.mjs
```

Nếu foundation guard fail thì **không sửa guard để cho qua**; báo leader vì đó là contract chung.

Làm pricing trước

Mở:

```
frontend/lib/pricing.ts
```

Hoàn thiện:

```ts
export type CartSize = 'S' | 'M' | 'L';

export const SIZE_SURCHARGE: Record<CartSize, number> = {
  S: 0,
  M: 5_000,
  L: 10_000,
};

export const TOPPING_SURCHARGE = 5_000;

export function calculateUnitPrice(
  basePrice: number,
  size: CartSize,
  toppings: string[],
): number {
  if (!Number.isInteger(basePrice) || basePrice < 0) {
    throw new Error('Invalid base price');
  }

  const uniqueToppings = [...new Set(toppings)];

  return (
    basePrice +
    SIZE_SURCHARGE[size] +
    uniqueToppings.length * TOPPING_SURCHARGE
  );
}

export function calculateLineTotal(
  unitPrice: number,
  qty: number,
): number {
  return unitPrice * qty;
}
```

Mục tiêu của file này **chỉ là UI preview**. Không dùng nó làm security boundary.

Viết Zustand Cart store

Mở:

```
frontend/store/cart.ts
```

Skeleton đề xuất:

```ts
'use client';

import { create } from 'zustand';
import {
  createJSONStorage,
  persist,
} from 'zustand/middleware';

export type CartSize = 'S' | 'M' | 'L';

export interface CartItem {
  productId: string;
  name: string;
  size: CartSize;
  toppings: string[];
  qty: number;
  unitPrice: number;
}

interface CartState {
  items: CartItem[];

  add: (item: CartItem) => void;

  remove: (
    productId: string,
    size: CartSize,
    toppings: string[],
  ) => void;

  setQty: (
    productId: string,
    size: CartSize,
    toppings: string[],
    qty: number,
  ) => void;

  clear: () => void;

  total: () => number;
  count: () => number;
```

```
}

function normalizeToppings(toppings: string[]) {
  return [...new Set(toppings)].sort();
}

function keyOf(item: {
  productId: string;
  size: CartSize;
  toppings: string[];
}) {
  return [
    item.productId,
    item.size,
    normalizeToppings(item.toppings).join(','),
  ].join('|');
}

export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      items: [],

      add: (incoming) => {
        const cleanItem: CartItem = {
          ...incoming,
          toppings: normalizeToppings(incoming.toppings),
          qty: Math.min(99, Math.max(1, incoming.qty)),
        };

        set((state) => {
          const key = keyOf(cleanItem);
          const existing = state.items.find(
            (item) => keyOf(item) === key,
          );

          if (!existing) {
            return {
              items: [...state.items, cleanItem],
            };
          }

          return {
            items: state.items.map((item) =>
              keyOf(item) === key
                ? {
                    ...item,
                    qty: Math.min(
                      99,
                      item.qty + cleanItem.qty,
                    ),
```

```
            }
          : item,
      ),
    };
  });
},

remove: (productId, size, toppings) => {
  const target = keyOf({
    productId,
    size,
    toppings,
  });

  set((state) => ({
    items: state.items.filter(
      (item) => keyOf(item) !== target,
    ),
  }));
},

setQty: (productId, size, toppings, qty) => {
  const target = keyOf({
    productId,
    size,
    toppings,
  });

  const nextQty = Math.min(
    99,
    Math.max(1, Math.trunc(qty)),
  );

  set((state) => ({
    items: state.items.map((item) =>
      keyOf(item) === target
        ? { ...item, qty: nextQty }
        : item,
    ),
  }));
},

clear: () => set({ items: [] }),

total: () =>
  get().items.reduce(
    (sum, item) =>
      sum + item.unitPrice * item.qty,
    0,
  ),
```

```
      count: () =>
        get().items.reduce(
          (sum, item) => sum + item.qty,
          0,
        ),
    }),
    {
      name: 'brewlite-cart-v1',
      version: 1,
      storage: createJSONStorage(
        () => localStorage,
      ),
      partialize: (state) => ({
        items: state.items,
      }),
    },
  ),
);
```

Zustand `persist` được thiết kế chính xác cho việc giữ state qua reload và hỗ trợ `localStorage`; việc dùng Client Component là phù hợp vì `localStorage` là browser API. [6]

Giao contract `add()` cho B

Người B vẫn sở hữu:

```
frontend/app/products/[id]/page.tsx
```

Guide B yêu cầu nút "Thêm vào giỏ" phải hoạt động. filecite turn0file15

C **không cần tự sửa file này trên branch C** nếu hai người đang code song song. Thay vào đó gửi B contract:

```
const add = useCartStore((state) => state.add);

add({
  productId: product.id,
  name: product.name,
  size,
  toppings,
  qty: 1,
  unitPrice: calculateUnitPrice(
    product.price,
    size,
    toppings,
  ),
});
```

Sau khi B gọi `add()`:

```
Product Detail
    ↓
Zustand
    ↓
localStorage
```

không gọi backend.

Viết trang `/cart`

Mở:

```
frontend/app/cart/page.tsx
```

Đây phải là Client Component vì có Zustand state và click handler; Next.js yêu cầu client boundary cho state, event handler và browser-only behavior. [1]

Skeleton:

```tsx
'use client';

import Link from 'next/link';
import { useCartStore } from '@/store/cart';

function formatVnd(value: number) {
  return new Intl.NumberFormat('vi-VN', {
    style: 'currency',
    currency: 'VND',
  }).format(value);
}

export default function CartPage() {
  const items = useCartStore((state) => state.items);
  const setQty = useCartStore((state) => state.setQty);
  const remove = useCartStore((state) => state.remove);

  const total = items.reduce(
    (sum, item) => sum + item.unitPrice * item.qty,
    0,
  );

  if (items.length === 0) {
    return (
      <main>
        <h1>Giỏ hàng</h1>
        <p>Giỏ hàng đang trống.</p>
```

```
        <Link href="/">Quay lại menu</Link>
      </main>
    );
  }

  return (
    <main>
      <h1>Giỏ hàng</h1>

      {items.map((item) => (
        <article
          key={`${item.productId}-${item.size}-${[
            ...item.toppings,
          ]
            .sort()
            .join(',')}`}
        >
          <h2>{item.name}</h2>

          <p>Size: {item.size}</p>

          <p>
            Topping:{' '}
            {item.toppings.length > 0
              ? item.toppings.join(', ')
              : 'Không'}
          </p>

          <p>{formatVnd(item.unitPrice)} / ly</p>

          <button
            type="button"
            disabled={item.qty <= 1}
            onClick={() =>
              setQty(
                item.productId,
                item.size,
                item.toppings,
                item.qty - 1,
              )
            }
          >
            -
          </button>

          <span>{item.qty}</span>

          <button
            type="button"
            disabled={item.qty >= 99}
            onClick={() =>
```

```
              setQty(
                item.productId,
                item.size,
                item.toppings,
                item.qty + 1,
              )
            }
          >
            +
          </button>

          <button
            type="button"
            onClick={() =>
              remove(
                item.productId,
                item.size,
                item.toppings,
              )
            }
          >
            Xóa
          </button>

          <p>
            Thành tiền:{' '}
            {formatVnd(
              item.unitPrice * item.qty,
            )}
          </p>
        </article>
      ))}

      <strong>
        Tổng cộng: {formatVnd(total)}
      </strong>

      <Link href="/checkout">
        Thanh toán
      </Link>
    </main>
  );
}
```

`<Link>` là primitive chính của Next.js cho client-side navigation giữa route; ở đây dùng nó cho `/checkout` đơn giản hơn gọi router thủ công. [7]

Làm Cart badge mà không phá layout

Tạo:

frontend/components/cart-link.tsx

```
'use client';

import Link from 'next/link';
import { useEffect, useState } from 'react';
import { useCartStore } from '@/store/cart';

export function CartLink() {
  const [mounted, setMounted] = useState(false);

  const count = useCartStore((state) =>
    state.items.reduce(
      (sum, item) => sum + item.qty,
      0,
    ),
  );

  useEffect(() => {
    setMounted(true);
  }, []);

  return (
    <Link href="/cart">
      Giỏ hàng ({mounted ? count : 0})
    </Link>
  );
}
```

Sau đó trong:

frontend/app/layout.tsx

thay link Cart tĩnh bằng:

```
import { CartLink } from '@/components/cart-link';
```

và:

```
<CartLink />
```

Cách tách này giữ phần lớn layout ở phía server nhưng cô lập phần cần Zustand/browser state vào Client Component. Next.js khuyến nghị chỉ đặt client boundary nơi thực sự cần interactivity/browser APIs. [1]

Không tự viết checkout

Khi bấm:

```
Thanh toán
```

C chỉ chuyển tới:

```
/checkout
```

Không tạo Order ngay ở `/cart`.

Không gọi:

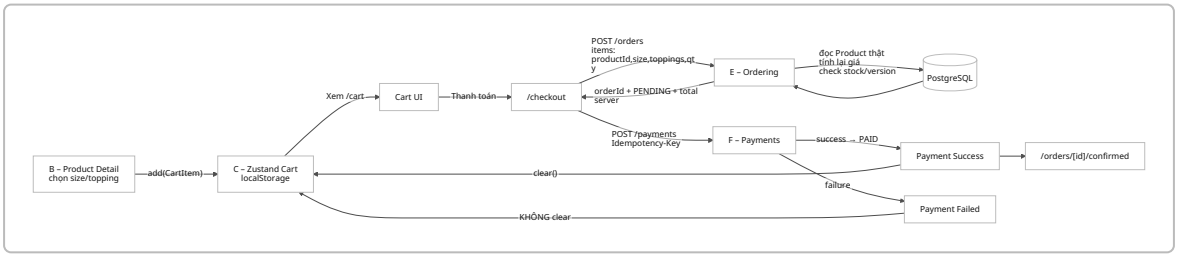
```
POST /payments
```

từ `/cart`.

Không clear giỏ khi người dùng chỉ mới mở checkout.

Guide F yêu cầu cart chỉ bị xóa khi payment thành công; nếu payment thất bại phải giữ nguyên để thử lại. filecite turn0file11

Luồng Cart → Ordering → Payments và HTTP contract



Luồng này tổng hợp đúng ownership từ guides C, E và F: Cart giữ state ở frontend; Ordering tạo đơn và kiểm tra stock/tính giá; Payments thực hiện mock payment/idempotency, clear cart chỉ sau thành công. filecite turn0file14 filecite turn0file12 filecite turn0file11

HTTP mẫu lấy Product

Cart không nhất thiết gọi API này trực tiếp; thông thường B gọi.

```
GET /products/PRODUCT_ID
```

Response tối thiểu theo contract Product của đề:

```
{
  "id": "PRODUCT_ID",
  "name": "Cappuccino",
  "price": 45000,
  "imageUrl": "/images/cappuccino.jpg",
  "stock": 20
}
```

Đề BrewLite yêu cầu `GET /products` và `GET /products/:id`, với Product tối thiểu gồm `id`, `name`, `price`, `imageUrl`, `stock`. filecite turn0file9

Frontend sau khi người dùng chọn:

```
M
Kem
qty = 1
```

có thể lưu:

```
{
  "productId": "PRODUCT_ID",
  "name": "Cappuccino",
  "size": "M",
  "toppings": ["CREAM"],
  "qty": 1,
  "unitPrice": 55000
}
```

`55.000` chỉ là **snapshot UI**.

HTTP mẫu tạo Order

Khi checkout bắt đầu, chuyển state Cart thành request tối giản:

```
POST /orders
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "items": [
    {
      "productId": "PRODUCT_ID",
      "size": "M",
      "toppings": ["CREAM"],
      "qty": 1
```

```
    }
  ]
}
```

Không gửi:

```
{
  "name": "Cappuccino",
  "unitPrice": 1000,
  "lineTotal": 1000,
  "total": 1000
}
```

như dữ liệu có quyền quyết định giá.

Ordering phải lấy Product thật và tính lại. Guide E quy định `POST /orders` nhận danh sách item, tính tổng và tạo Order `PENDING`; endpoint đặt đơn yêu cầu đăng nhập. filecite turn0file12

Response đề xuất:

```
{
  "orderId": "ORDER_ID",
  "status": "PENDING",
  "total": 55000
}
```

Từ thời điểm này, UI phải dùng `total` của server cho thanh toán, không dùng tổng Cart cũ.

HTTP mẫu Payment

Sau khi Order tồn tại:

```
POST /payments
Authorization: Bearer <accessToken>
Idempotency-Key: 54f7029c-4ff1-4e97-a183-1ddb35008501
Content-Type: application/json
```

```
{
  "orderId": "ORDER_ID",
  "method": "WALLET"
}
```

Guide F quy định payment nhận `orderId`, `method`, hỗ trợ `WALLET/CARD` và dùng `Idempotency-Key` để request lặp không tạo nhiều payment. filecite turn0file11

Thành công:

```
{
  "orderId": "ORDER_ID",
  "status": "PAID"
}
```

Frontend F gọi:

```
useCartStore.getState().clear();
```

rồi chuyển:

```
/orders/ORDER_ID/confirmed
```

Thất bại:

```
hiển thị lỗi
giữ nguyên Cart
cho phép retry
```

đúng với guide Payments. filecite turn0file11

Kiểm thử, nghiệm thu và bằng chứng bắt buộc

Unit test phải chứng minh gì?

Guide C gốc chưa yêu cầu framework test cụ thể, nhưng GUIDE-V2 nên tự động hóa phần logic quan trọng. Foundation đã có lint/build frontend, vì vậy Cart nên bổ sung unit test cho pricing/store và browser E2E cho luồng thực tế.

Nếu project chưa có frontend unit-test runner, dùng Vitest; nếu chưa có browser E2E, Playwright là lựa chọn hợp lý cho Next.js. Playwright hỗ trợ TypeScript, chạy browser thật và có test runner/report/trace dành cho end-to-end testing. [8]

File đề xuất:

```
frontend/tests/unit/pricing.test.ts
frontend/tests/unit/cart.test.ts
frontend/e2e/cart.spec.ts
frontend/playwright.config.ts
```

Unit cases cho pricing

Case	Input	Expected
Size S	35k, S, 0 topping	35k
Size M	35k, M, 0 topping	40k
Size L	35k, L, 0 topping	45k
1 topping	35k, S, 1 topping	40k
2 toppings	35k, M, 2 toppings	50k
Duplicate topping	`["CREAM","CREAM"]`	tính 1 lần
qty	unit 50k × 3	150k

Các expected value bám theo pricing rule trong guide C gốc. filecite turn0file14

Unit cases cho Cart store

Ít nhất:

```
CART-U01
Cart ban đầu rỗng.

CART-U02
add() món mới → 1 dòng.

CART-U03
add() cùng product + size + toppings
→ không tạo dòng mới, tăng qty.

CART-U04
cùng product nhưng size khác
→ 2 dòng.

CART-U05
cùng product + size nhưng toppings khác
→ 2 dòng.

CART-U06
setQty() cập nhật đúng.

CART-U07
qty không xuống dưới 1.

CART-U08
qty không vượt 99.

CART-U09
remove() chỉ xóa đúng variant.
```

```
CART-U10
total() tính đúng nhiều dòng.

CART-U11
count() = tổng qty.

CART-U12
clear() → items = [].
```

E2E cases

`frontend/e2e/cart.spec.ts` cần tối thiểu:

ID	Kịch bản
`CART-E01`	Mở Product Detail → chọn size/topping → Add → badge tăng
`CART-E02`	Vào `/cart` thấy đúng name/size/topping/price
`CART-E03`	Refresh browser → cart vẫn còn
`CART-E04`	`+/-` thay qty và cập nhật total
`CART-E05`	Xóa item → item biến mất
`CART-E06`	Empty cart hiển thị trạng thái trống
`CART-E07`	"Thanh toán" → `/checkout`
`CART-E08`	Hai cấu hình cùng Product được giữ thành hai dòng
`CART-E09`	Payment fail → Cart còn nguyên, thực hiện khi tích hợp F
`CART-E10`	Payment success → Cart bị clear, thực hiện khi tích hợp F

Playwright chính thức chạy được bằng `npx playwright test`; có thể chạy riêng Chromium và có HTML report/trace phục vụ evidence PR. [9]

Lệnh chạy trước khi tạo PR

Tại frontend:

```
cd frontend
npm ci
npm run lint
npm run build
npm test
```

Nếu script unit test đặt là Vitest:

```
npm run test -- --run
```

E2E:

```
npx playwright test e2e/cart.spec.ts --project=chromium
```

Có thể xem report:

```
npx playwright show-report
```

Playwright cung cấp lệnh chạy test và HTML report chính thức theo các command trên. [10]

Sau đó quay về root:

```
cd ..
node scripts/verify-foundation.mjs
docker compose up -d --build --wait
docker compose ps
```

Foundation trước đó đã chứng minh stack có thể build/start thành công; Cart PR không được làm mất trạng thái đó. filecite turn0file8

Manual smoke test bắt buộc

Dùng Chrome/Edge:

```
1. Mở /
2. Mở 1 sản phẩm.
3. Chọn M.
4. Chọn 1 topping.
5. Add.
6. Kiểm tra badge = 1.
7. Add lại cùng cấu hình.
8. Kiểm tra badge = 2.
9. Vào /cart.
10. Refresh.
11. Item vẫn còn.
12. Nhấn +.
13. Total thay đổi.
14. Nhấn -.
15. Total quay lại.
16. Nhấn Thanh toán.
17. URL thành /checkout.
```

Sau đó DevTools → Application → Local Storage phải thấy key:

```
brewlite-cart-v1
```

Zustand persist dùng storage key theo `name`, và state persisted có thể sống qua page reload. [2]

Evidence người C phải nộp cho leader

Không chỉ nhắn "em làm xong".

Thư mục evidence nên có:

```
evidence/C-cart/
├── 01-foundation-guard.log
├── 02-frontend-lint.log
├── 03-frontend-build.log
├── 04-cart-unit-tests.log
├── 05-cart-e2e-tests.log
├── 06-docker-compose-ps.log
├── 10-product-add.png
├── 11-cart-page.png
├── 12-cart-after-refresh.png
├── 13-cart-qty-total.png
├── 14-cart-localstorage.png
└── README.md
```

Screenshot nên thể hiện:

```
Product Detail có size/topping
Badge đúng qty
/cart có item
Refresh không mất
Total đúng
Checkout navigation hoạt động
```

Acceptance criteria, PR checklist và các giả định còn mở

Definition of Done của C

Task C chỉ được đánh dấu **DONE** khi toàn bộ bảng dưới đạt.

Tiêu chí	Bắt buộc
Add từ Product Detail vào Cart	✅
Cùng variant được merge qty	✅
Variant khác không bị merge	✅
Zustand persist localStorage	✅

Tiêu chí	Bắt buộc
Refresh không mất Cart	✅
`+/-` hoạt động	✅
Xóa một item hoạt động	✅
Empty state có UI	✅
Giá S/M/L đúng	✅
Topping +5.000 mỗi loại	✅
Total nhiều item đúng	✅
Badge = tổng qty	✅
Checkout → `/checkout`	✅
Client không quyết định giá Order	✅
Không tạo Cart table/API	✅
Unit test xanh	✅
E2E Cart xanh	✅
`npm run lint` xanh	✅
`npm run build` xanh	✅
Foundation guard xanh	✅
Docker stack vẫn chạy	✅

Sáu tiêu chí cốt lõi đầu bài của Cart là thêm sản phẩm, persist localStorage, thay đổi/xóa số lượng, tính tổng, badge và chuyển checkout. Các mục còn lại trong bảng là hardening GUIDE-V2 để module ghép được an toàn với B/E/F. 🗄filecite🗄turn0file14🗄

PR checklist cho developer C

Trước khi mở PR `feature/c-cart → main`:

- [ ] Chỉ sửa file thuộc ownership C hoặc file integration đã thống nhất.
- [ ] Không sửa Prisma schema.
- [ ] Không tạo endpoint `/cart`.
- [ ] Không sửa Orders/Payments service.
- [ ] Không duplicate Product logic của B.
- [ ] `CartItem` không chứa dữ liệu authentication/payment.
- [ ] `unitPrice` có comment rõ là display-only.
- [ ] Request tạo Order không tin `unitPrice` / `total` client.
- [ ] Cart chỉ clear sau payment success.
- [ ] Payment fail giữ Cart.
- [ ] `npm run lint` PASS.
- [ ] `npm run build` PASS.
- [ ] unit PASS.

- [ ] E2E Cart PASS.
- [ ] `verify-foundation.mjs` PASS.
- [ ] Evidence đã commit hoặc attach PR.
- [ ] Có ít nhất một người review.

Definition of Done chung của đề BrewLite cũng yêu cầu code build được, đạt acceptance criteria, commit rõ ràng, được review và có hướng dẫn chạy. filecite turn0file9

Những việc C tuyệt đối không làm

```
❌ Tạo bảng Cart trong Prisma.

❌ Tạo POST /cart chỉ vì "thấy app bán hàng thường có".

❌ Trừ stock lúc add vào cart.

❌ Tin unitPrice trong localStorage.

❌ Gửi total frontend rồi lưu thẳng vào Order.

❌ Clear Cart ngay khi POST /orders thành công.

❌ Clear Cart khi payment thất bại.

❌ Sửa MockGateway.

❌ Sửa Order State Machine.

❌ Sửa Payment idempotency.

❌ Tự merge code vào Product Detail của B mà không phối hợp B.
```

Stock/state-machine thuộc E, còn payment/idempotency thuộc F theo guides tương ứng. filecite turn0file12 filecite turn0file11

Open questions và assumptions cần ghi trong manifest

Không cần dừng code để hỏi; V2 dùng các assumption sau cho tới khi Master Contract/leader đổi.

Vấn đề chưa khóa hoàn toàn	Assumption GUIDE-V2 C
Identifier Product là UUID hay string khác	Cart dùng `string`, không tự parse
Tên topping chuẩn	Cart giữ `string[]`; lấy giá trị từ UI B
Topping có được lặp lại không	Không; normalize unique
Same Product khác size	Dòng khác

Vấn đề chưa khóa hoàn toàn	Assumption GUIDE-V2 C
Same Product cùng size nhưng topping khác	Dòng khác
Same variant add lần hai	Merge qty
Max quantity	`99` mỗi variant
Badge	Tổng `qty`
Cart database	Không có
Cart API	Không có
Cart tồn tại bao lâu	Theo localStorage browser
Multi-tab realtime sync	Không thuộc MVP
Login trước hay sau Cart	Được add khi chưa login; login bắt buộc trước `POST /orders`
Product hết hàng sau khi add	E phát hiện tại create-order
HTTP stock conflict	Checkout giữ Cart và báo thử lại
Coupon	G/F checkout xử lý; không thuộc Cart
Loyalty	G xử lý; không thuộc Cart
Giá authoritative	Backend E
`unitPrice` persisted	UI snapshot, không security-sensitive

Việc cho phép người dùng dựng Cart trước khi đăng nhập phù hợp với luồng đề bài, trong đó login có thể xảy ra sau bước xem Cart và trước thanh toán/tạo đơn. filecite turn0file9

Manifest đề xuất

`BrewLite_GUIDE-V2_C-Cart_2026-10-15.manifest.json`:

```
{
  "guide": "C - Cart",
  "version": "GUIDE-V2",
  "date": "2026-10-15",
  "project": "BrewLite",
  "foundation": "V1",
  "gate01": "PASS",
  "scope": [
    "Task 5 - Cart"
  ],
  "ownedFiles": [
    "frontend/store/cart.ts",
    "frontend/lib/pricing.ts",
    "frontend/app/cart/page.tsx",
    "frontend/components/cart-link.tsx"
```

```
  ],
  "sharedEditFiles": [
    "frontend/app/layout.tsx"
  ],
  "integrationDependencies": {
    "B": "Product Detail calls Cart.add()",
    "E": "Ordering recalculates final price and validates stock",
    "F": "Payments calls Cart.clear() only after success"
  },
  "persistence": {
    "type": "localStorage",
    "key": "brewlite-cart-v1",
    "version": 1
  },
  "pricing": {
    "frontendRole": "preview only",
    "authoritativeModule": "E - Ordering",
    "sizeSurcharge": {
      "S": 0,
      "M": 5000,
      "L": 10000
    },
    "toppingSurcharge": 5000
  },
  "assumptions": [
    "qty range is 1..99",
    "duplicate toppings are removed",
    "same product/size/toppings merges quantity",
    "multi-tab synchronization is outside MVP"
  ],
  "requiredChecks": [
    "foundation guard",
    "frontend lint",
    "frontend build",
    "cart unit tests",
    "cart browser e2e",
    "docker compose smoke test"
  ]
}
```

Trạng thái sẵn sàng giao việc: `READY FOR IMPLEMENTATION`.

Người C có thể bắt đầu từ `frontend/lib/pricing.ts` → `frontend/store/cart.ts` → `/cart` → badge → integration với B → tests. Phần quan trọng nhất khi review là bảo đảm **Cart chỉ giữ dữ liệu và giá preview ở client; Ordering mới là nơi quyết định giá, tồn kho và tạo Order; Payments mới là nơi quyết định khi nào được clear Cart**. Cách chia này giữ đúng Task 5 của đề, tránh giẫm ownership B/E/F và giảm nguy cơ ghép code cuối Sprint bị vỡ. filecite turn0file14 filecite turn0file15 filecite turn0file12 filecite turn0file11

---

1 Getting Started: Server and Client Components | Next.js
https://nextjs.org/docs/app/getting-started/server-and-client-components?utm_source=chatgpt.com

2 4 6 persist - Zustand Docs
https://zustand.docs.pmnd.rs/reference/middlewares/persist.html?utm_source=chatgpt.com

3 Validation | NestJS - A progressive Node.js framework
https://docs.nestjs.com/v11/techniques/validation?utm_source=chatgpt.com

5 Transactions and batch queries (Reference) (Prisma ORM v6) | Prisma Documentation
https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions?utm_source=chatgpt.com

7 Components: Link Component | Next.js
https://nextjs.org/docs/app/api-reference/components/link?utm_source=chatgpt.com

8 9 10 Installation | Playwright
https://playwright.dev/docs/intro?utm_source=chatgpt.com